

An explicit degree-3059 Littlewood multiple of

$$1 + x^2 + x^3 + x^8 + x^9 + x^{10}$$

Xiaojian Zeng

PBC School of Finance, Tsinghua University, 100190

xiaojianzeng@foxmail.com

Abstract

We give an explicit Littlewood polynomial of degree 3059 divisible in $\mathbb{Z}[x]$ by $1 + x^2 + x^3 + x^8 + x^9 + x^{10}$. Its quotient has degree 3049 and height 15. The proof checks all 3060 integer coefficient sums; descending monic division gives a second verification. A complete compact encoding of the signs and a short verifier are included, with the full quotient array supplied separately. This settles the degree-3059 existence question for this fixed factor. No minimum-degree classification is claimed.

1 Statement and context

A Littlewood polynomial is a polynomial all of whose coefficients, from the constant term through its leading term, belong to $\{-1, 1\}$. Set

$$P(x) = 1 + x^2 + x^3 + x^8 + x^9 + x^{10}, \quad S = \{0, 2, 3, 8, 9, 10\}.$$

The question addressed here is whether P has a Littlewood multiple of degree exactly 3059.

Theorem 1. *There exists $Q \in \mathbb{Z}[x]$ of degree 3049 such that*

$$P(x)Q(x) = \sum_{j=0}^{3059} \varepsilon_j x^j, \quad \varepsilon_j \in \{-1, 1\} \quad (0 \leq j \leq 3059).$$

The certificate in Appendix A specifies such a multiple explicitly.

Drungilas, Jankauskas, Junevičius, Klebonas and Šiurys [1, Theorem 6 and Table 3] list this polynomial among the exceptional factors in their computation, with period 1020. Idris and Sac-Épée [2, Sections 3–5] give the necessary degree condition $\deg L = 1020k - 1$ and report exclusions at degrees 1019 and 2039; their conclusion leaves the least feasible $k > 2$ undetermined. Theorem 1 supplies an explicit positive certificate at $k = 3$. The reported earlier exclusions are not re-certified here, and they are not assumptions of the theorem.

The contribution is this concrete multiple, not a new convolution or division method. A bounded comparison of the two cited primary papers and degree-specific searches on 7 September 2026 found no equivalent degree-3059 certificate in the inspected sources. This comparison does not establish priority or exhaustive literature coverage. For broad adjacent context on exact finite polynomial certificates, see Zeng [3]; that reference concerns a different polynomial problem and supplies no premise of Theorem 1.

2 Certificate and proof

Write $q_0, \dots, q_{3049}$ for the integer array in the accompanying `witness.json`, in ascending exponent order, and put

$$Q(x) = \sum_{t=0}^{3049} q_t x^t.$$

The same array can be reconstructed solely from the complete sign encoding printed in Appendix A: with $q_t = 0$ for $t < 0$, use

$$q_j = \varepsilon_j - q_{j-2} - q_{j-3} - q_{j-8} - q_{j-9} - q_{j-10}, \quad 0 \leq j \leq 3049. \quad (1)$$

Thus the certificate does not require repeating the discovery search.

Proof of Theorem 1. Use the explicit signs in Appendix A and the quotient above. Its entries are integers, $q_0 = 1$, and $q_{3049} = -1$. Extend $q_t = 0$ also for $t > 3049$. Multiplication by the displayed P gives

$$[x^j](PQ) = q_j + q_{j-2} + q_{j-3} + q_{j-8} + q_{j-9} + q_{j-10}. \quad (2)$$

Exact addition for every $j = 0, \dots, 3059$ gives $[x^j](PQ) = \varepsilon_j \in \{-1, 1\}$. The full finite check is implemented in the printed verifier: it reconstructs all 3050 quotient entries and forms all 3060 product coefficients, including the final ten coefficients not prescribed by recurrence (1). No index is sampled or omitted. Its output is recorded below. Since $\deg P = 10$ and $q_{3049} = -1$, the product has degree exactly 3059 and no coefficient above that degree. Hence PQ is the required Littlewood polynomial. $\square$

For this particular witness, the maximum absolute quotient coefficient is 15, and the product has 1512 positive and 1548 negative coefficients. Its constant coefficient is 1 and its leading coefficient is -1 . These are finite observations about the displayed witness.

The verifier also performs descending long division of the sign polynomial by P . Monicity ensures that this uses only integer subtraction and multiplication: at each step the current leading coefficient determines the next quotient entry. The remainder is zero, and the recovered quotient agrees entry by entry with the one obtained from (1). This supplies a check with the reverse arithmetic order.

Discovery and limitations. The certificate was found by a SAT search in the restricted family $R = (1 + x^2)Q$ with $\max_j |[x^j]R| \leq 4$; the resulting R has height exactly 4. No completeness assertion about this restricted family is needed or made. The proof depends only on the explicit integer identity, not on solver correctness, timing, or infeasibility of another search. We do not independently exclude smaller degrees, classify all multiples, or determine the minimum degree.

Computational assistance and reproducibility. An AI assistant helped derive search reductions, write scripts, and prepare this manuscript. HiGHS and CaDiCaL were used during discovery; the successful witness came from the restricted SAT search. Certificate verification used exact Python integer arithmetic. A separate AI review recomputed the product and quotient; this is not journal peer review or proof-assistant formalization. The supplement contains `witness.json`, `littlewood.hex`, and `verify_hex.py`. Running `python3 verify_hex.py`, with the latter two files in the same directory, requires only the Python 3 standard library. The full array and compact encoding represent the same integer witness.

References

- [1] P. Drungilas, J. Jankauskas, G. Junevičius, L. Klebonas and J. Šiurys, *On certain multiples of Littlewood and Newman polynomials*, 2018, [arXiv:1801.07179](https://arxiv.org/abs/1801.07179).
- [2] M. Idris and J.-M. Sac-Épée, *On the existence of Newman and Littlewood multiples for certain integer polynomials*, 2026, [arXiv:2607.15520v1](https://arxiv.org/abs/2607.15520v1).
- [3] Z. Zeng, *An Eight-Element Counterexample to Real-Rootedness of Preorder Support Polynomials*, 2026, SSRN preprint, [doi:10.2139/ssrn.7385158](https://doi.org/10.2139/ssrn.7385158).

A Complete sign certificate

Concatenate the following 17 lines, each of 45 hexadecimal digits, to obtain $H = h_0 \cdots h_{764}$. For a digit with value $h_i = 8b_{i,0} + 4b_{i,1} + 2b_{i,2} + b_{i,3}$, where each $b_{i,r}$ is 0 or 1, set $\varepsilon_{4i+r} = 1 - 2b_{i,r}$ for $0 \leq r < 4$. Thus each digit is read from its most significant bit to its least significant bit, and exponents increase from left to right and from top to bottom. The 765 digits encode all 3060 signs without padding.

```
482eeda7e711bb5398a31e886118f3bcaf3631ccb3ed1
5b0528c9a9cc3e95a99ef8e67019237f91afa8a791f9d
2700ce11f2071397609da2e6b2fd81e15b661ab07e55f
c4e284334c3a06dc0c6ccfe63de46610fe4fe60fdc731
ade011c0f8c93c899c93da335335e3bfb1fe75f841bf4
3733de036033e0f7f9cd077849b8e6ac1f940d25ec136
59ce809c032ce3f98f7919843f93f983e28ff1de0ceb9
8f632eb0bbca6711191a4013f073e79f04f7213f24648
f889bbcbe22bf25dcc36cb380867007f67741fc613c4e
98b08c4f61bf8c6701fe619f075c039e10c3bfb6bf30e
c2e6903f339c1d844f8019671fcc7bc9e721f833cc1f1
47f8c3de788166c19e27f44da5397c7c4f760ffce5e70
67086e67bc06c067c08e339a0efb8fc1cdb0af281a4bd
8249087103fb862d8ff31ee0db22c689f625d194d07e4
22e222c65c057f929e459e14fc8620e7cf3e09ec11ef4
3e0d6b50de10c457e45a241bb0b2073187fecee83b207
f4904c63fc994199e8ee10f32ba03becafc4ff30fe43f
```

Save these lines as `littlewood.hex`. The following is the complete file `verify_hex.py`; run it without Python's `-O` option so that assertions are enabled.

```
"""Verify the complete certificate using Python 3 integer arithmetic."""
from pathlib import Path

H = "".join(Path(__file__).with_name("littlewood.hex").read_text().split())
assert len(H) == 765 and set(H) <= set("0123456789abcdef")
L = [1 - 2 * ((int(c, 16) >> r) & 1)
      for c in H for r in (3, 2, 1, 0)]
S = (0, 2, 3, 8, 9, 10)
q = []
for j in range(3050):
    q.append(L[j] - sum(q[j-s] for s in S[1:] if j >= s))

product = [0] * 3060
for j, a in enumerate(q):
    for s in S:
        product[j+s] += a
assert product == L
assert (q[0], q[-1], L[0], L[-1]) == (1, -1, 1, -1)
assert max(map(abs, q)) == 15
assert (L.count(1), L.count(-1)) == (1512, 1548)

# Independent descending division by the monic polynomial P.
remainder, quotient = L.copy(), [0] * 3050
for j in range(3049, -1, -1):
    a = remainder[j+10]
    quotient[j] = a
    for s in S:
        remainder[j+s] -= a
assert not any(remainder) and quotient == q
print("PASS: 3060 signs; degree(Q)=3049; degree(PQ)=3059;")
print("height(Q)=15; exact product and zero remainder verified.")
```

The exact output is:

```
PASS: 3060 signs; degree(Q)=3049; degree(PQ)=3059;
height(Q)=15; exact product and zero remainder verified.
```